

Web Scraping Introduction

Web scraping is a technique or program used to copy and gather information from websites. In R, web scraping is used for creating data set from the web for statistical analysis.

To initiate webscraping, the Rvest package [1] needs to be installed and loaded:

```
install.packages("rvest") # for the first time  
library(rvest)
```

Then use 'read_html' to export the HTML source code of the webpage for scraping:

```
input_url <- "https://example.com"  
htmlcode <- read_html(input_url)
```

We need to first identify where to extract the data from the source. HTML text contents are mostly structured with the following elements:

<div>, , <p>, <h1>, <td>, , etc...

Some contents are generated by JavaScript, which is very challenging to extract as the code is no longer in HTML, the Rvest package will not be able to harvest the data from scripted code [2]. Let's focus on pure HTML content for better understanding.

In order to extract the content quoted from the HTML elements, we can use the html_node (for once) / html_nodes (for all) to tell R where the content is located. There are two ways to locate the content [3]:

1. By CSS Selector: e.g. `html_nodes( htmlcode, "span.class")`
2. By XPath Selector e.g. `html_nodes( htmlcode, xpath="//div[@class='...']/.../span")`

To have better illustration of applying both methods, I will go through an example.

Web scrapping on AutoTrader search results

Recently my friend was planning to buy a car, he is looking at the Mini Cooper. Autotrader.ca is one of the seller platforms for dealers to release their first/second-hand vehicles. To give him a better picture of the market, I planned to do a web scraping on current selling prices in Vancouver.

Let's start with copying the URL of the search result (first page):

```
input_url <- "  
https://www.autotrader.ca/cars/bc/burnaby/?rcp=0&rcs=0&prx=100&prv=British%20Columbia&loc=V5A%20S6&hprc=True&wcp=True&sts=New-Used&inMarket=basicSearch&make=MINI" [4]
```

We define the variables according to what data we'd like to extract from the webpage.

```
desc <- c() # Description of the vehicle (Brand, Model, Feature etc)  
odom <- c() # Odometer of the vehicle  
location <- c() # Location of the vehicle  
price <- c() # Current list price of the vehicle
```

Let's take a deep look on the search results. You may notice that the total number of listings displayed is more than the number we've selected. For instance, you will get 26 listings for setting 15 to display per page. This is because it shows "Priority Listings" (duplicates) and "From Areas You Might Also Consider" (either duplicates or distance over 100km) blended with the "All Listings". This is still manageable if we can find out an indicator for the "All Listings".

For this example, right-click the text area and go to Inspect, we can identify a listing to be in "All Listings" only if it starts with a "<div>" with CSS class="col-xs-12 result-item enhanced organic-qa listing-redesign-dt".

Now we'll try implementing the html_nodes to locate the contents needed. Let's try with the CSS selector method first:

The 1st node extracts all the "All Listings" to avoid the extra listings, and the 2nd node extracts the node matches the CSS style provided. Also note that all spaces must be replaced by a full stop as that's how the CSS style was classified:

```
desc <- htmlcode %>% html_nodes("div.col-xs-12.result-item.enhanced.organic-qa.listing-redesign-dt") %>% html_node("span.title-with-trim")
```

We can try to compare this with the XPath method:

XPath search by beginning with an identifier and then go through a fixed pathway until it hits the last element. XPath might just be easier if the selections are uniquely represented by an id or class, we can then directly copy the XPath from the element during inspection. However, it's usually not the case. In this example, we can extract the full XPath:

```
"//html/body/div[3]/div/div[2]/div[1]/div/div[2]/div/div[4]/div[2]/div[19]/div[2]/div[1]/div/a/div[1]/div[2]/div[2]/div/h2/span/span"
```

"div[19]" is where the "All Listings" identifier located, so we want to search the "<div>" with the specified class:

```
desc <- htmlcode %>% html_nodes(xpath="//div[@class='col-xs-12 result-item enhanced organic-qa listing-redesign-dt']/div[2]/div[1]/div/a/div/div[2]/div[2]/div/h2/span/span")
```

Now let's look at extracting the mileage values:

```
odom <- htmlcode %>% html_nodes(xpath="//div[@class='col-xs-12 result-item enhanced organic-qa listing-redesign-dt']/div[2]/div[1]/div/a/div/div[2]/div[1]/div[2]/div/span")
```

This extraction generated a shorter list than the other variables because odometer readings on new cars are not mandatory! To avoid "missing" the missing data, we can first generate a list of "All Listings" nodes, then do the actual node search. That way the `html_node()` will return "NA" for the missing data.

This also shows XPath approach is prone to the indexing of the elements, for instance, if we are searching on "div[2]" but one of the listings is missing "div[1]", then "div[2]" will become "div[1]" and show us the wrong data. The following will be the amendment on the extraction of mileage:

```
odom <- htmlcode %>% html_nodes(xpath="//div[@class='col-xs-12 result-item enhanced organic-qa listing-redesign-dt']/div[2]/div[1]/div/a/div/div[2]/div[1]") %>%  
html_node("span.odometer-proximity")
```

Iterate over Pagination

Despite knowing how to extract data from a webpage, a single page is usually not enough to display all search results. While most web designs use pagination to divide the results, there are some webpages using infinite scroll that requires Javascript to load the next results, which we are not going to cover in this article.

The pagination settings are usually shown on the URL, for instance,

```
input_url <- "  
https://www.autotrader.ca/cars/bc/burnaby/?rcp=0&rsc=0&prx=100&prv=British%20Columbia&loc=V5A%20S6...
```

Where "rcp" is the number of listings per page, and "rsc" is the total number of listings from the previous pages; One can switch the numbers to verify its variables. Here we can create a base URL with 100 per page, split the url into two to substitute the increment of rsc during the iteration:

```
base_url <- input_url %>% str_replace("rcp=.+&rsc","rcp=100&rsc") %>%  
str_split_fixed("rsc=[^&]+",2)
```

Now we can count the number of pages with the total number of listings:

```
htmlcode <- read_html(input_url)  
  
nresults <- htmlcode %>% html_node("span.title-count") %>% html_text() %>%  
str_replace_all("[^\\d]","") %>% as.integer()  
  
iterations <- ceiling(nresults/100)
```

And start iterating through the pagination:

```
for (i in 0:(iterations-1)) {  
  # Construct the URL for the current page, substituting "rsc=i*100"  
  url <- str_c(base_url[1],base_url[2],sep=str_c("rsc=",i*100))  
  
  # Read HTML content from the current page  
  htmlcode <- read_html(url)  
  
  # We can append the previous page results onto the new lists:
```

```
desc <- htmlcode %>% html_nodes("div.col-xs-12.result-item.enhanced.organic-
qa.listing-redesign-dt") %>% html_node("span.title-with-trim") %>% html_text() %>%
append(desc)

location <- htmlcode %>% html_nodes("div.col-xs-12.result-item.enhanced.organic-
qa.listing-redesign-dt") %>% html_node("span.proximity-text.overflow-ellipsis") %>%
html_text() %>% append(location)

odom <- htmlcode %>% html_nodes("div.col-xs-12.result-item.enhanced.organic-
qa.listing-redesign-dt") %>% html_node("span.odometer-proximity") %>% html_text()
%>% append(odom)

price <- htmlcode %>% html_nodes("div.col-xs-12.result-item.enhanced.organic-
qa.listing-redesign-dt") %>% html_node("span.price-amount") %>% html_text() %>%
append(price)

}
```

After the iteration, we should have the search results' data needed. We just need to tidy them up:

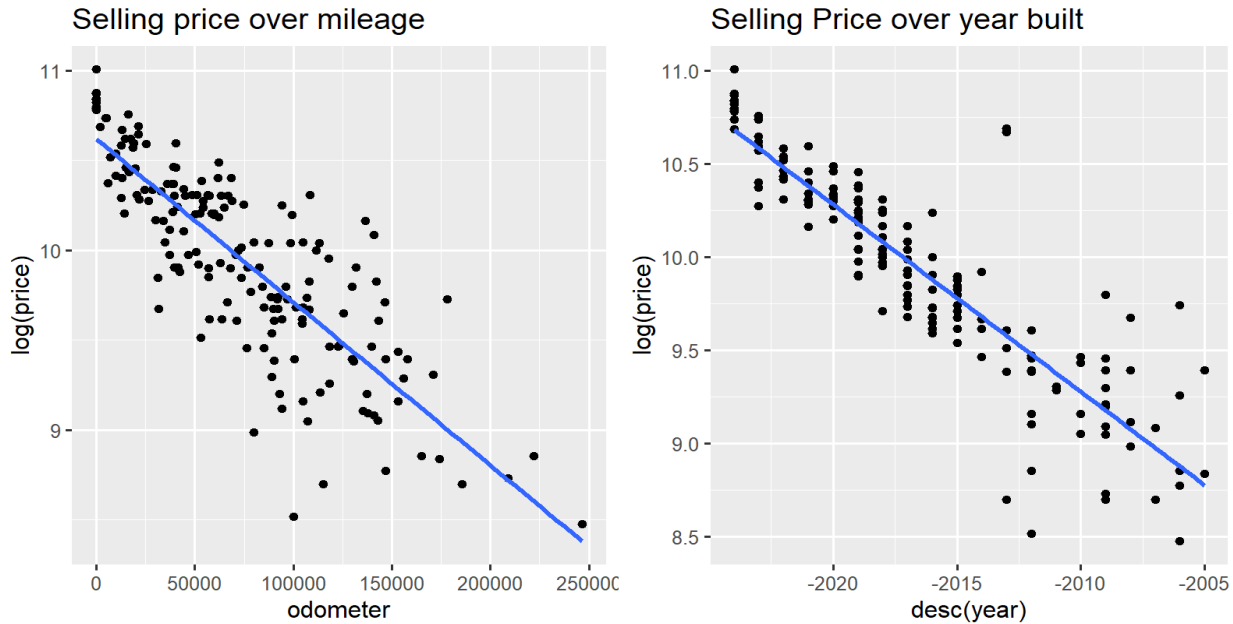
```
desc <- desc %>% str_remove_all("\\s\\s")      # Remove excess whitespaces
odom <- odom %>% str_remove_all("[^\\d]")      # Integers only
price <- price %>% str_remove_all("[^\\d]")    # Integers only
```

Combine the data into a table:

```
tbl <- tibble("brand"=str_split_fixed(desc,"\\s",3)[,2],
"description"=str_split_fixed(desc,"\\s",3)[,3],
"location"=location,"year"=as.integer(str_split_fixed(desc,"\\s",3)[,1]),
"odometer"=as.integer(odom),"price"=as.integer(price))
```

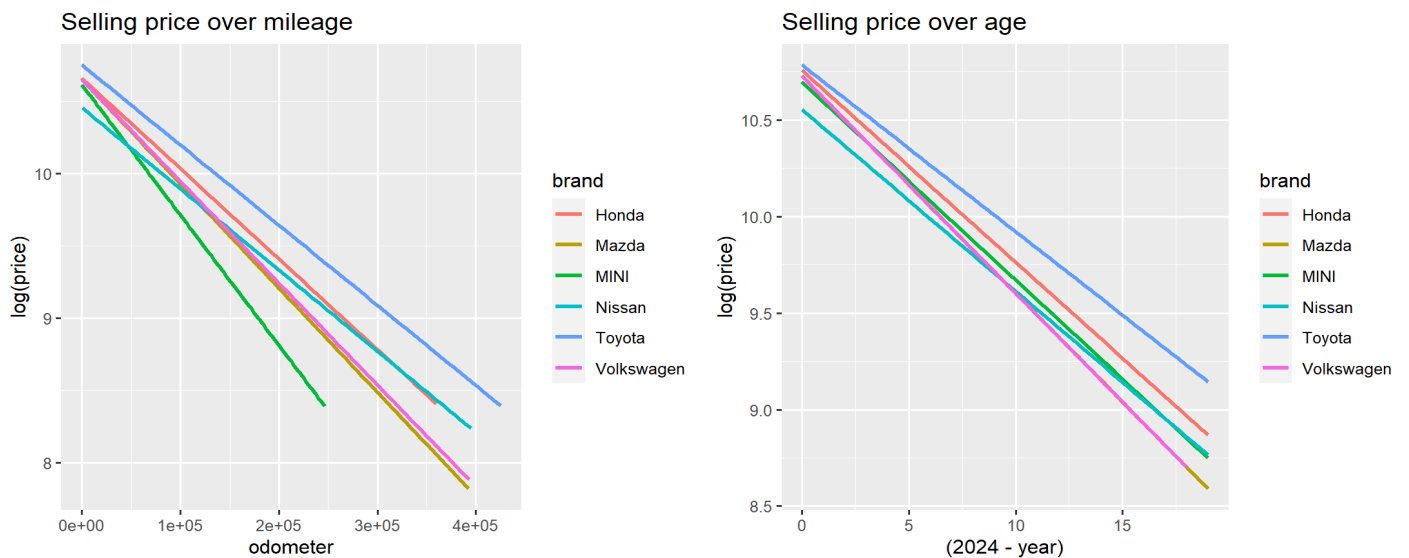
Data Analysis

Finally, we can plot price over mileage and manufacturing year:



This can be a reference for a fair/average selling price, if my friend has \$10,000 budget, he may expect it to be a 2011 Mini Cooper with a mileage of around 160,000 km.

We can also loop the above with different models to generate the depreciation rate (slope) of reselling value on different brands:



Conclusion

Throughout this example, we've explored the fundamental concepts of web scraping, from locating the data using CSS selectors and XPath, to iterating the data extraction through pagination. Even though the R code was constructed specifically on a webpage, the technique to extract data is applicable to most of the selling or advertisement platforms such as online marketplaces and job postings.

In conclusion, web scraping in R offers a powerful means of extracting real-time data from online source for statistical analysis. With a wider range and more robust web-scraping R code, one can collect data from different webpages or even on a regular basis for time-series analysis. This can provide us a broader scope on studying a particular market to make better decisions.

References

- [1] H. Wickham, "rvest: Easily Harvest (Scrape) Web Pages," 2024. [Online]. Available: <https://rvest.tidyverse.org/>. [Accessed 2024 March 25].
- [2] F. Schouwenaars, "Short tutorial on scraping Javascript generated data with R using PhantomJS.," DataCamp, March 2015. [Online]. Available: <https://www.datacamp.com/tutorial/scraping-javascript-generated-data-with-r>. [Accessed 25 March 2024].
- [3] "html_nodes function - RDocumentation," [Online]. Available: https://www.rdocumentation.org/packages/rvest/versions/0.3.4/topics/html_nodes. [Accessed 25 March 2024].
- [4] "New and Used Cars for Sales.," [Online]. Available: www.autotrader.ca/. [Accessed 25 March 2024].